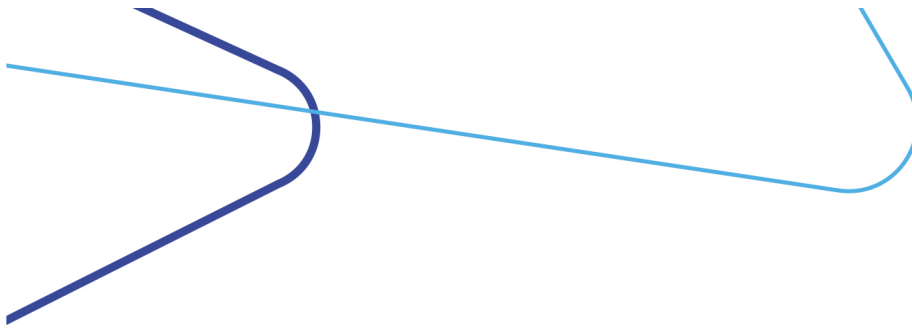




Case Técnico: Desenvolvedor Web e Infraestrutura JR

PVT Software & Serviços

Esse case faz parte do nosso processo seletivo. A ideia não é montar um sistema completo nem entregar tudo funcionando, é entender como você pensa: **como modela um problema, organiza uma solução e toma decisões técnicas.**

Capriche mais no raciocínio do que na quantidade de código.

Contexto

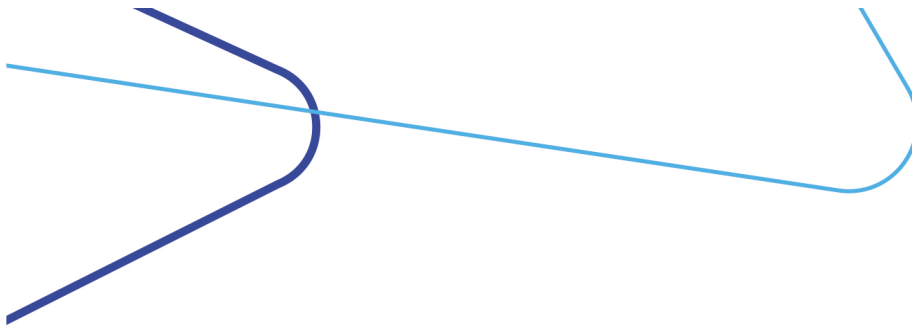
A PVT é parceira TOTVS e atua, entre outras frentes, alocando analistas em projetos de clientes e gerenciando esses projetos de ponta a ponta. Para isso mantemos um Portal de Gestão (web) que centraliza projetos, alocações, agendas, apontamentos de horas, propostas e faturamento, tendo o TOTVS RM como ERP de retaguarda (fonte oficial dos dados).

No dia a dia, um ponto crítico é o acompanhamento de saúde dos projetos: cada analista aponta as horas trabalhadas, e a coordenação precisa enxergar, por projeto, quanto foi vendido, planejado e realizado, qual o saldo de horas restante e quais projetos estão críticos (estourando horas ou com entrega física atrasada em relação ao financeiro).

O desafio

Imagine que a PVT vai construir (ou reconstruir) o módulo "Painel de Projetos" do portal. Sua missão é projetar tecnicamente esse módulo: desde a modelagem dos dados até a API e as regras de negócio, considerando que o dado-fonte vem de um ERP externo (o RM).

Você não precisa integrar com o TOTVS RM de verdade — pode tratá-lo como uma "caixa



externa" e simular/mockar os dados que viriam dele.

Parte 1 — Design e Arquitetura (obrigatória)

Entregue um documento (pode ser um README.md) cobrindo:

1. Modelagem do domínio.

Quais entidades você usaria (ex.: Projeto, Cliente, Analista, Alocação, Apontamento...), seus principais atributos e como elas se relacionam. Um diagrama simples (ER ou caixas e setas) ajuda bastante.

2. Arquitetura da solução.

Como você organizaria as camadas (front, API/back, acesso a dados) e onde entraria a comunicação com o ERP externo. Um diagrama de fluxo é bem-vindo.

3. Design da API.

Liste os principais endpoints REST que o painel precisaria (recurso, verbo HTTP e o que cada um retorna). Ex.: listar projetos com saldo, registrar um apontamento, etc.

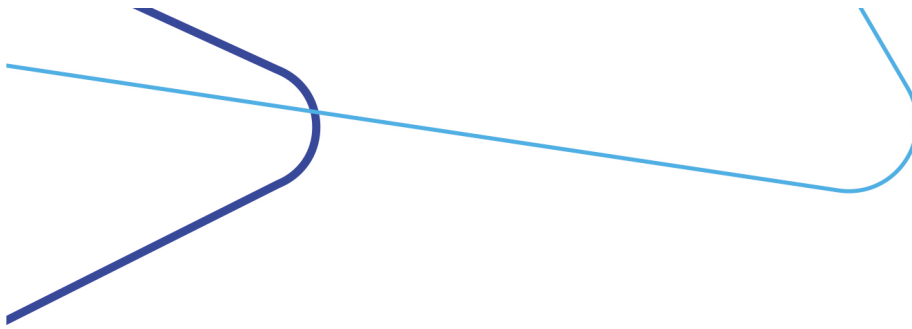
4. Regras de negócio.

Descreva como você calcularia:

- Saldo de horas de um projeto;
- O percentual de avanço;
- A classificação de status (ex.: saudável / atenção / crítico) e o que torna um projeto "crítico".

5. Decisões e trade-offs.

Por que você escolheu essa modelagem/arquitetura? O que considerou e descartou?



6. Como eu evoluiria isso.

Pensando em escala (centenas de projetos, relatórios pesados vindos do ERP, vários acessos simultâneos): o que você faria para garantir performance e resiliência? O que fazer quando o ERP estiver fora do ar ou lento?

Não existe resposta única certa. Queremos ver critério, clareza e consistência nas escolhas.

Parte 2 — Implementação (opcional / bônus)

Se quiser e tiver tempo, implemente uma fatia mínima do que você desenhou, não precisa cobrir tudo:

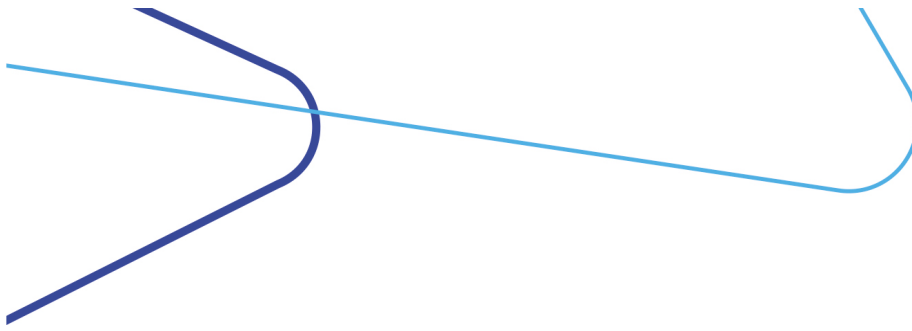
- 1 endpoint que lista projetos com o saldo de horas calculado, ou 1 endpoint que registra um apontamento;
- a regra de cálculo do saldo/status funcionando;
- (se sobrar fôlego) uma tela simples consumindo esse endpoint.

Pode usar dados mockados/seed. Vale mais um fluxo pequeno e bem-feito do que muita coisa pela metade.

Stack

Use o que você domina melhor. Mas, se possível, alinhe ao nosso ambiente, porque é nele que você trabalharia:

- Front: React (ou Angular) + TypeScript, com atenção a responsividade.
- Back/BFF: Node.js (ou .NET/C#, se for mais sua praia).



- Banco: qualquer relacional (SQL Server, PostgreSQL, MySQL...).

Escolher uma stack próxima da nossa conta a favor, mas não é eliminatório. Explicar bem as escolhas conta mais.

Entrega

- Um repositório Git (GitHub) com o documento da Parte 1 e, se fizer, o código da Parte 2 — ou, se preferir não usar repo, um documento (.md ou PDF) com o design.
- Um README curto com: como rodar (se houver código), e um resumo das suas decisões.
- Prazo sugerido: até 5 dias corridos a partir do recebimento.

Como vamos avaliar

Transparência total: o peso maior está no raciocínio, não na quantidade de código. Vamos olhar principalmente:

- Modelagem do domínio e coerência das entidades;
- Organização da arquitetura e separação de responsabilidades;
- Clareza do design de API e das regras de negócio;
- Visão de evolução/escala e tratamento de falhas;
- Comunicação (quão claro está o seu documento).

Depois da entrega, faremos uma conversa técnica (+/- 15 min) onde você apresenta a solução e a gente discute algumas decisões juntos. **Essa conversa é parte importante da avaliação.**